

Lab 4: Using an external library & the Dining Philosophers

In this lab, you will use an external (image processing) library and solve a classic problem from Computer Science: the Dining Philosophers. In this lab, you **MUST** create a directory “lab4/” and put all your code inside it. You **MUST present** this lab **AND** submit on Canvas all your working directory as a unique compressed file “lab4.tar.gz” using “tar” and “gzip” in Linux.

1) Using an external library

On our Linux system, we will install and use the famous computer Vision and Image Processing library: OpenCV (<https://opencv.org/>). Please follow the different steps, and ensure that there is no error at one step before going to the next one.

a) Install the library on your system:

```
$ sudo apt update
```

```
$ sudo apt install libopencv-dev
```

These lines will first update/refresh the list of packages available for your Linux system online. Then, you will install “libopencv-dev”, the package containing the libraries and header files to work with OpenCV. Of course, apt will install many more packages because the package “libopencv-dev” needs a lot of dependencies to work: packages are built upon other needed packages.

To list the dependencies of any package, you can first search for the package:

```
$ apt list libopencv-dev
```

```
os2024@os2024-virtualbox:~$ apt list libopencv-dev
Listing... Done
libopencv-dev/noble,now 4.6.0+dfsg-13.lubuntu1 amd64 [installed]
```

Once you know the name of the package you are looking for, print the list of dependencies with apt show:

```
os2024@os2024-virtualbox:~$ apt show libopencv-dev
Package: libopencv-dev
Version: 4.6.0+dfsg-13.lubuntu1
Priority: optional
Section: universe/libdevel
Source: opencv
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Debian Science Team <debian-science-maintainers@lists.alioth.debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 359 kB
Depends: libopencv-calib3d-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-contrib-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-core-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-dnn-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-features2d-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-flann-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-highgui-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-imgcodecs-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-imgproc-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-ml-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-objdetect-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-photo-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-shape-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-stitching-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-superres-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-video-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-videoio-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-videostab-dev (= 4.6.0+dfsg-13.lubuntu1), libopencv-viz-dev (= 4.6.0+dfsg-13.lubuntu1), pkg-config, libc6 (>= 2.34), libgcc-s1 (>= 3.3.1), libopencv-calib3d406t64 (>= 4.6.0+dfsg), libopencv-contrib406t64 (>= 4.6.0+dfsg), libopencv-core406t64 (>= 4.6.0+dfsg), libopencv-dnn406t64 (>= 4.6.0+dfsg), libopencv-features2d406t64 (>= 4.6.0+dfsg), libopencv-highgui406t64 (>= 4.6.0+dfsg), libopencv-imgcodecs406t64 (>= 4.6.0+dfsg), libopencv-imgproc406t64 (>= 4.6.0+dfsg), libopencv-video406t64 (>= 4.6.0+dfsg), libstdc++6 (>= 11), libliblevdev-dev
Recommends: opencv-data, libopencv-java (= 4.6.0+dfsg-13.lubuntu1)
Suggests: opencv-doc
Homepage: https://opencv.org
Download-Size: 84.2 kB
APT-Manual-Installed: yes
APT-Sources: http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages
Description: development files for opencv
This is a metapackage providing development package necessary for development of OpenCV (Open Computer Vision).

The Open Computer Vision Library is a collection of algorithms and sample code for various computer vision problems. The library is compatible with IPE (Intel's Image Processing Library) and, if available, can use IPP (Intel's Integrated Performance Primitives) for better performance.

OpenCV provides low level portable data types and operators, and a set of high level functionalities for video acquisition, image processing and analysis, structural analysis, motion analysis and object tracking, object recognition, camera calibration and 3D reconstruction.
```

b) List of libraries

To know the list of available OpenCV libraries installed on your system, you must run:

```
$ ldconfig -v | grep opencv
```

You will then get a list of available OpenCV libraries (there are many of them) installed on your system, ready to be used.

```
libopencv_flann.so.406 -> libopencv_flann.so.4.6.0
libopencv_video.so.406 -> libopencv_video.so.4.6.0
libopencv_dpm.so.406 -> libopencv_dpm.so.4.6.0
libopencv_barcode.so.406 -> libopencv_barcode.so.4.6.0
libopencv_imgcodecs.so.406 -> libopencv_imgcodecs.so.4.6.0
libopencv_objdetect.so.406 -> libopencv_objdetect.so.4.6.0
libopencv_highgui.so.406 -> libopencv_highgui.so.4.6.0
libopencv_dnn_objdetect.so.406 -> libopencv_dnn_objdetect.so.4.6.0
libopencv_freetype.so.406 -> libopencv_freetype.so.4.6.0
```

c) An image to test OpenCV

We need an image to use OpenCV, I suggest downloading the same image (but you can try any other image you want):

```
$ mkdir lab4/
```

```
$ cd lab4/
```

```
$ wget
```

```
https://ju.se/images/18.25d2a00c18e4b8edf8e36924/1711454221086/x1920p/JU\_SPARK\_170913-0357\_2000x500.jpg
```

```
$ mv JU_SPARK*.jpg ju.jpg
```

```
$ ls -l
```

```
os2024@os2024-virtualbox:~/lab4$ wget https://ju.se/images/18.25d2a00c18e4b8edf8e36924/1711454221086/x1920p/JU_SPARK_170913-0357_2000x500.jpg
--2024-11-18 14:50:16-- https://ju.se/images/18.25d2a00c18e4b8edf8e36924/1711454221086/x1920p/JU_SPARK_170913-0357_2000x500.jpg
Resolving ju.se (ju.se)... 193.10.208.54
Connecting to ju.se (ju.se)|193.10.208.54|:443... connected.
HTTP request sent, awaiting response... 200
Length: 126782 (124K) [image/jpeg]
Saving to: 'JU_SPARK_170913-0357_2000x500.jpg'

JU_SPARK_170913-0357_2000x500.jpg 100%[=====] 123,81K --KB/s in 0,002s

2024-11-18 14:50:16 (63,1 MB/s) - 'JU_SPARK_170913-0357_2000x500.jpg' saved [126782/126782]

os2024@os2024-virtualbox:~/lab4$ ls -l
total 124
-rw-rw-r-- 1 os2024 os2024 126782 mar 26 2024 JU_SPARK_170913-0357_2000x500.jpg
os2024@os2024-virtualbox:~/lab4$ mv JU_SPARK*.jpg ju.jpg
os2024@os2024-virtualbox:~/lab4$ ls -l
total 124
-rw-rw-r-- 1 os2024 os2024 126782 mar 26 2024 ju.jpg
os2024@os2024-virtualbox:~/lab4$
```

d) A first program to test OpenCV

The OpenCV library is written in C++ (which is object-oriented C). We didn't learn C++ and won't learn it. So, just copy and paste the following code in your preferred text editor, save it as "image1.cpp" (the .cpp extension is used for c plus plus -cpp- file):

```
1 // C++ OpenCV program
2 #include <iostream>
3 #include <opencv2/opencv.hpp>
4 using namespace cv;
5 using namespace std;
6
7 int main(int argc, char** argv)
8 {
9     // Read the image file
10    Mat image = imread("../ju.jpg", IMREAD_GRAYSCALE);
11
12    // Error Handling
13    if (image.empty()) {
14        cout << "Image File Not Found" << endl;
15
16        // wait for any key press
17        cin.get();
18        return -1;
19    }
20
21    // Show Image inside a window with the name provided
22    imshow("My JU image", image);
23
24    // Wait for any keystroke
25    waitKey(0);
26    return 0;
27 }
28
29
```

To compile this code, you MUST use g++ (instead of gcc) because the code is written in C++. You also need to specify where the included files "opencv.hpp" and "imgproc.hpp" are in your filesystem.

Find these files using:

```
$ find /usr -name imgproc.hpp
```

Or

```
$ locate imgproc.hpp
```

When you know the directory, add it to the following command line (-I for included header files directory, -L for library directory, -l for library names):

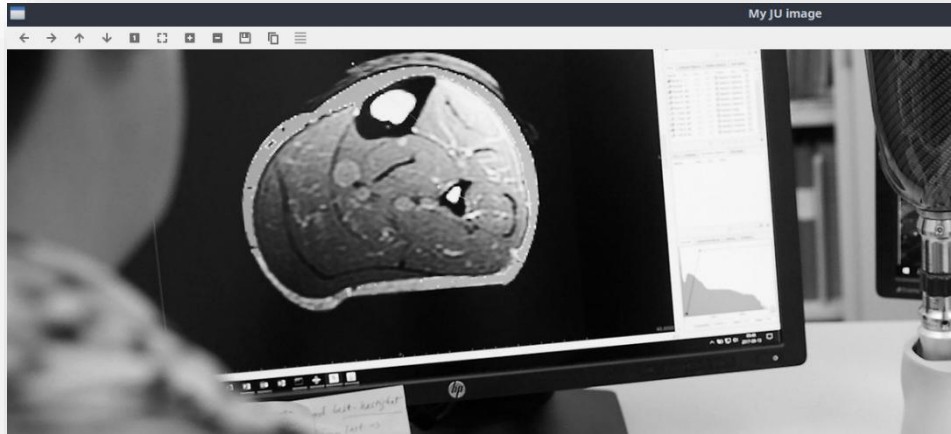
```
$ g++ image1.cpp -o image1 -I/the/path/you/found/for/your/included/header/files
-L/usr/lib/x86_64-linux-gnu -lopencv_core -lopencv_highgui -lopencv_imgcodecs
```

```
os2024@os2024-virtualbox:~/lab4$ ls -l
total 156
-rwxrwxr-x 1 os2024 os2024 24584 nov 19 07:43 image1
-rw-rw-r-- 1 os2024 os2024 559 nov 19 07:37 image1.cpp
-rw-rw-r-- 1 os2024 os2024 126782 mar 26 2024 ju.jpg
os2024@os2024-virtualbox:~/lab4$
```

Once compiled with success, run the executable file image1:

```
$ ./image1
```

You will obtain a window with the image converted to GRAYSCALE. Half of the image is shown in this figure (the image has a large width, not fitting on my screen):



Now that you understand how things work, copy your file “image1.cpp” to “image2.cpp”, add a Canny function call to detect the contours (objects edges) in the image:

...

```
// Canny edge/contours detection  
Canny(image, image, 100, 300, 3);
```

...

Compile the file, be careful, you must know that the Canny function is in the include file headers “opencv2/imgproc.hpp” and the library file is “libopencv_imgproc.so”.

Run the image2 executable and you will obtain the contours/edges of the image:



If you can use your webcam within your Linux OS in VirtualBox, make a live video version of the contours/edges detection system on the video coming from your camera. You can use AI to help you to complete this task. If you do so, cite your resources in your code.

2) The Dining Philosophers

The Dining Philosophers is a problem proposed and solved by Dijkstra in 1965 to illustrate mutual exclusion to avoid deadlocks, in the case of multiple threads trying to access multiple resources at the same time. It is a classical problem that is very famous in Computer Science.

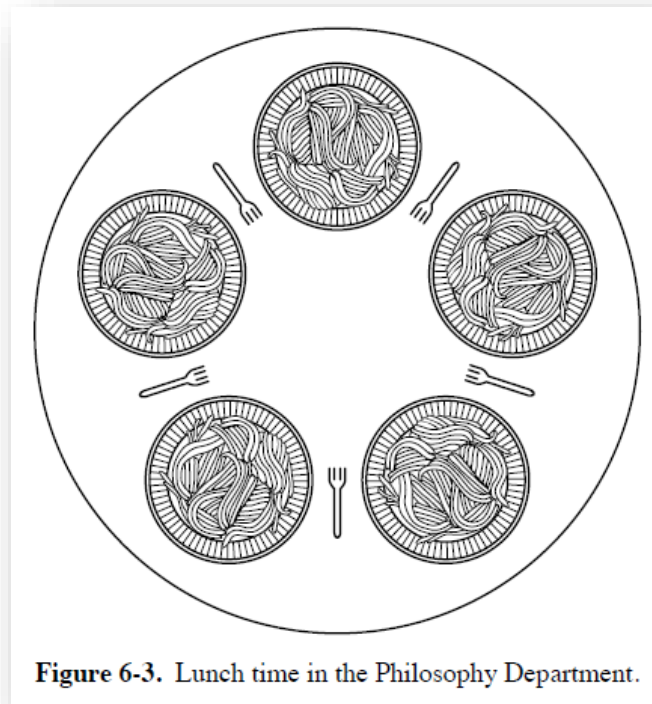


Figure 6-3. Lunch time in the Philosophy Department.

Source: (Tanenbaum-Bos, "Modern Operating Systems, 5th edition", pp. 442-443, 2024)

You consider five philosophers who want to eat some (delicious) Italian pasta. To eat, a philosopher needs two forks, the fork on the left of his plate and the one on the right. Of course, if every philosopher takes, for instance, the left fork at the same time, there is a deadlock happening. In this exercise, you must propose a solution to this problem and explain it to the teacher when presenting.

You must have five philosophers (so five plates with pasta) and five forks. You must use threads and avoid deadlocks. To solve the problem, you can use any resource/tool you want: AI chatbots, books, articles... You must understand the problem and the solution, the teacher will ask you questions during your presentation. You **MUST** create a running C code to solve this problem that will print the state of each philosopher when it changes from thinking to eating and reverse. Each philosopher must eat at least 10 times before your program stops.

This is the end. In this lab, you learned to install and use an external library and solve the dining philosophers problem. Congratulations, grattis!